

PROGRAMMING LANGUAGES

SIMPLE IMPERATIVE LANGUAGE IN ANTLR

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026



© Michele Pasqua, 2026

SIMPLE IMPERATIVE LANGUAGE



BEYOND EXPRESSIONS

Interpreter `ImpIntp` for a simple imperative language

- ▶ We already have assignments
- ▶ Now we add other classic imperative constructs, like conditionals and loops

```
if (x == 5) { ... }  
else { ... }
```

```
while (x < 5)  
{ ... }
```

Programs are sequences of semicolon-separated **commands** `com1 ; com2 ; ...`

- ▶ We need **booleans** (true/false) for testing conditionals

BEYOND EXPRESSIONS (CONT'D)

What do programs evaluate to?

```
i = 0 ; while (i < 9) { i = i + 1 }
```

$\Downarrow$ $\llbracket \text{ImpIntp} \rrbracket$
no value

Programs now **transform** memories

$$[i \mapsto 3] \xrightarrow{\llbracket \text{ImpIntp} \rrbracket(i = 0 ; \text{while } (i < 9) \{ i = i + 1 \})} [i \mapsto 9]$$

LANGUAGE SEMANTICS

Memory $\sigma \in \mathbb{M} \rightarrow \mathbb{V}$ mapping variables to values

- Values now can be integers or booleans

$$\text{ASSIGN} \frac{\langle \sigma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \sigma, x = \text{exp} \rangle \longrightarrow \langle \sigma, x = \text{exp}' \rangle}$$

$$\text{ASSIGNE} \frac{-}{\langle \sigma, x = v \rangle \longrightarrow \langle \sigma[v/x], \epsilon \rangle}$$

Boolean expressions

$$\text{NOT} \frac{\langle \sigma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \sigma, \text{not exp} \rangle \longrightarrow \langle \sigma, \text{not exp}' \rangle}$$

$$\text{NOTE} \frac{-}{\langle \sigma, \text{not } b \rangle \longrightarrow \langle \sigma, \bar{b} \rangle} \quad b \in \{\text{true}, \text{false}\}$$

LANGUAGE SEMANTICS (CONT'D)

Conditionals

$$\text{If} \frac{\langle \sigma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \sigma, \text{if } (\text{exp}) \{ \text{com} \} \rangle \longrightarrow \langle \sigma, \text{if } (\text{exp}') \{ \text{com} \} \rangle}$$

$$\text{IfT} \frac{-}{\langle \sigma, \text{if } (\text{true}) \{ \text{com} \} \rangle \longrightarrow \langle \sigma, \text{com} \rangle}$$

$$\text{IfF} \frac{-}{\langle \sigma, \text{if } (\text{false}) \{ \text{com} \} \rangle \longrightarrow \langle \sigma, \epsilon \rangle}$$

What happens when `exp` does not evaluate to `true` or `false`?

- The interpreter should raise a `runtime` error

LANGUAGE SEMANTICS (CONT'D)

Finally, sequences of commands

$$\text{SEQ} \frac{\langle \sigma, \text{com}_1 \rangle \longrightarrow \langle \sigma', \text{com}'_1 \rangle}{\langle \sigma, \text{com}_1 ; \text{com}_2 \rangle \longrightarrow \langle \sigma', \text{com}'_1 ; \text{com}_2 \rangle}$$

$$\text{SEQE} \frac{-}{\langle \sigma, \epsilon ; \text{com}_2 \rangle \longrightarrow \langle \sigma, \text{com}_2 \rangle}$$

Evaluation stops when `com` yields an empty configuration $\langle \sigma, \epsilon \rangle$

ADDING FEATURES



EXTEND THE SIMPLE IMPERATIVE LANGUAGE

Add support for string values ("ci ao", "2.34", ...) and the operators

- ▶ `print` displaying a string to standard output
- ▶ `□:□` concatenating two strings

Only strings can be displayed, other values must be converted via the operator

- ▶ `toStr(□)` taking an expression `□` and returning its string representation

The output operator `escapes` quotes, backslash, and non-printable characters

```
print "hello \\t world \\\" \\\\\";           // prints on screen: hello      world " \
print toStr(x);                             // if x takes 8.4 print on screen: 8.4
print "h" : toStr(3) : "11" : toStr(4 mod 2); // print on screen: h3110

print 4                                     // print an error since 4 is not a string value
```

EXTEND THE SIMPLE IMPERATIVE LANGUAGE (CONT'D)

Add to the language

- ▶ The support for inline comments starting with `// ...`
- ▶ An `if` `else` conditional statement and a command `nop` that does nothing

```
x = 5;                      // a program example
tmp = x;
f = 1;
while (tmp > 1) {
  f = f * x;
  tmp = tmp - 1
};
print "The factorial of " : toString(x) : " is " : toString(f)
```

```
// equivalent to -->
if (x)
{ x = x + 1 }
```

```
// <-- equivalent to
if (x) { x = x + 1 }
else { nop }
```

HINTS

You may need to extend the value class hierarchy

To deal with string escaping, look at the `StringEscapeUtils` library

- ▶ The language escaping is Java-like
- ▶ Import `commons-text` from Apache with Maven

```
<dependencies>          <!-- to add to the pom.xml Maven configuration file -->
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-text</artifactId>
    <version>1.15.0</version>
  </dependency>
</dependencies>
```

Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

You are free to:

 **Share** – Copy, distribute, and display the work

Under the following terms:

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.